# Vacation-rental marketplace — production architecture

The submitted clone is only the front-end slice (static React app, no backend). This diagram represents the production-scale marketplace it would sit in.

request path · background events · 1 numbers follow one request

## 1 · FRONTEND

**Client** — browser (desktop web)

1

**DNS + WAF** — routing · basic protection

2

**CDN / edge cache** — static assets + images served close to users

3

**React web app** — static build, cached on the CDN

images

**Image optimization** — smaller resized / WebP images less bandwidth, faster pages

Scaling: the CDN caches static assets and images close to users; smaller images cut load time.

4

## 2 · BACKEND

stateless services, run in containers

**Listings** — details

**Search** — queries

**Availability** — calendar

**Booking** — reservations

**Users / Auth** — accounts, roles

5

**API gateway** — auth (JWT) · rate limits routing

Scaling: services are stateless, so scale by adding more identical instances behind a load balancer.

6

read / write

cache

images

events

## ASYNC · EVENT PROCESSING

**Kafka** — event stream / async event backbone booking · listing · image events

**Consumers / workers** — notifications · search-index updates media processing · analytics

Decouples services from background work.

Consumers scale independently as events grow.

queries

index updates

## 3 · STORAGE

resized images

**PostgreSQL** — primary + read replicas users, listings, bookings

**Redis cache** — frequently used data, sessions sits in front of the database

**Object storage** — listing images (originals) served through the CDN

Scaling: move reads to replicas and Redis; consider partitioning only if that is not enough.

## 4 · SEARCH

**Search index** — listing search + filters add replicas or shards as it grows

Scaling: keeps search load off the database.

## 5 · DEPLOYMENT

**CI/CD** — test → build container image preview per pull request

**Container platform** — AWS ECS, or Kubernetes as an option containerized backend services

**Load balancer + health checks** — traffic goes only to healthy instances; add more to scale

**Monitoring** — logs · metrics · basic tracing alerts on errors / latency

Scaling: build container → deploy → health checks → load balancer → identical stateless instances; add more when load grows.